Concept Formation by View Application

A second domain on shared foundations in CogLean

Jeff Shrager

November 2026

Draft for comment — not for circulation

Concept Formation by View Application

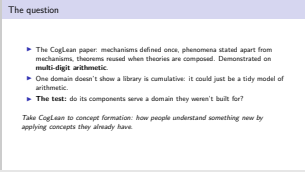


This talk goes with a draft paper of the same name, which is not for circulation and hasn't been carefully checked. It follows the CogLean paper, which introduced the library and the method using multi-digit arithmetic. This one is not an introduction to CogLean. It asks whether the library carries over to a completely different area of cognition, concept formation, and what it brings with it when it does.

The question

- The CogLean paper: mechanisms defined once, phenomena stated apart from mechanisms, theorems reused when theories are composed. Demonstrated on **multi-digit arithmetic**.
- One domain doesn't show a library is cumulative: it could just be a tidy model of arithmetic.
- **The test:** do its components serve a domain they weren't built for?

Take CogLean to concept formation: how people understand something new by applying concepts they already have.



Here's the question. The first paper argued that if you define mechanisms once, state phenomena apart from any mechanism, and prove consequences rather than simulate them, then theoretical results can accumulate, and it showed that on multi-digit arithmetic. But one domain doesn't show a library is cumulative. A library built around arithmetic might just be a well-organized model of arithmetic. The real test is whether its pieces serve a domain they weren't designed for. So we took it to concept formation: how people make sense of something new by applying concepts they already have.

Forming concepts from old ones

- Concepts live inside theories (Murphy & Medin 1985); they change by restructuring (Carey 1985; Rumelhart & Norman 1976).
- Analogy maps a known structure onto a new one (Gentner 1983); frames supply defaults (Minsky 1975).
- Learners build mental models of devices (Kieras & Bovair 1984).
- **View application** (Shrager 1987): a learner exploring a device with no instructions (Shrager & Klahr 1986) revises its theory by merging in a *view*, a schema of a familiar kind of thing.

Left open in 1987: **which** view is brought in, and **when** the theory is revised.

Forming concepts from old ones

- Concepts live inside theories (Murphy & Medin 1985); they change by restructuring (Carey 1985; Rumelhart & Norman 1976).
- Analogy maps a known structure onto a new one (Gentner 1983); frames supply defaults (Minsky 1975).
- Learners build mental models of devices (Kieras & Bovair 1984).
- **View application** (Shrager 1987): a learner exploring a device with no instructions (Shrager & Klahr 1986) revises its theory by merging in a view, a schema of a familiar kind of thing.

Left open in 1987: **which** view is brought in, and **when** the theory is revised.

Concept formation has been described in many ways. Concepts are embedded in theories that make them coherent. Children's concepts change by restructuring, not just by adding facts. Analogy maps the structure of something known onto something new, and frames supply defaults. People learning a device build a mental model of it. View application, from my 1987 paper, is a computational version of part of this. A learner exploring a device with no instructions has a theory of it, and changes that theory by applying a view: a schema of a familiar kind of thing, like a device that stores a program and runs it. The view is merged into the theory, overriding it where they conflict. That account left two things open: which view a learner brings in, and when the theory gets revised. Both, it turns out, were already in the library.

View application in CogLib

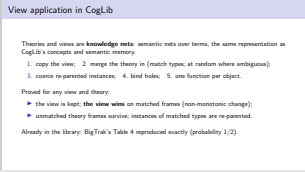
Theories and views are **knowledge nets**: semantic nets over terms, the same representation as CogLib's concepts and semantic memory.

1. copy the view; 2. merge the theory in (match types; at random where ambiguous);
3. coerce re-parented instances; 4. bind holes; 5. one function per object.

Proved for any view and theory:

- the view is kept; **the view wins** on matched frames (non-monotonic change);
- unmatched theory frames survive; instances of matched types are re-parented.

Already in the library: BigTrak's Table 4 reproduced exactly (probability $1/2$).



In CogLib, theories and views are knowledge nets: semantic nets whose nodes are terms, with frames, slots and patterns. It's the same representation the library uses for concepts and semantic memory, so view application doesn't bring a representation of its own. The process has the 1987 paper's five steps: copy the view, merge the theory into it, coerce re-parented instances, bind holes, and give each object a single function. Ambiguity is resolved at random, so applying a view gives a distribution over theories. For any view and theory, we prove the view is kept, that where both speak, the view wins, which is a non-monotonic change, that unmatched parts of the theory survive, and that instances get re-parented. And the library already reproduces the 1987 paper's BigTrak example exactly, half the time, which is what its random tie-breaking implies.

A concept result already there: number concepts

- Prototype concepts (*even*, *multiple of 3*) combined by selective modification (Smith et al. 1988), graded membership (Hampton 2007): 14 is an *even multiple of 3* to degree $\approx .80$. Over-extension.
- Apply an **ARITHMETIC view**:
  - *even multiple of 3* becomes exactly divisibility by 6, for every number;
  - no over-extension anywhere; **typicality unchanged**, for every number.
- So typicality and membership **dissociate**, as Armstrong, Gleitman & Gleitman (1983) found for *odd number*; impossible before the view.

A concept result already there: number concepts

- Prototype concepts (*even, multiple of 3*) combined by selective modification (Smith et al. 1988), graded membership (Hampton 2007): 14 is an *even multiple of 3* to degree ≈ .80. Over-extension.
- Apply an **ARITHMETIC view**:
  - *even multiple of 3* becomes exactly divisibility by 6, for every number;
  - no over-extension anywhere; **typicality unchanged**, for every number.
- So typicality and membership **dissociate**, as Armstrong, Gleitman & Gleitman (1983) found for *odd number*; impossible before the view.

There's already a concept-formation result in the library. Number concepts like even and multiple of three, learned as prototypes and combined by selective modification with graded membership, over-extend: fourteen counts as an even multiple of three to degree about point eight. Apply an arithmetic view, which replaces how the concept frame categorizes, and even multiple of three becomes exactly divisibility by six, for every number. Over-extension disappears everywhere, but typicality is unchanged, for every number. So typicality and membership come apart, which is what Armstrong and the Gleitmans found for odd number: graded typicality in a concept with all-or-none membership. Before the view that's provably impossible. A change of theory, not of examples, turned a prototype concept into a definitional one.

Which view? Retrieval through the shared memory

- Already: **spreading activation** (Collins & Loftus 1975). The same "+" key: *adds* with probability 2/3 for a naive learner; *appends a step* with 3/5 for one whose theory has a memory.
- New: **retrieval**. A cue (a key's label, the display) is coded and recognized by a memory, through CogLib's `Retriever` interface, the one the arithmetic model used. The stored view is applied.

For any memory:

- a cue confused with another brings in the other's view;
- a memory with the law of **determinacy** brings in one view per cue, at every attempt.

EPAM's net has determinacy at every setting; ACT-R's memory at none (the memory matrix).

Which view? Retrieval through the shared memory

- Already: **spreading activation** (Collins & Loftus 1975). The same "+" key: *adds* with probability 2/3 for a naive learner; *appends a step* with 3/5 for one whose theory has a memory.
- New: **retrieval**. A cue (a key's label, the display) is coded and recognized by a memory, through CogLib's `Retriever` interface, the one the arithmetic model used. The stored view is applied.

For any memory:

- a cue confused with another brings in the other's view;
- a memory with the law of **determinacy** brings in one view per cue, at every attempt.

EPAM's net has determinacy at every setting; ACT-R's memory at none (the memory matrix).

So, which view? The library already had one selector: spreading activation, where the same plus key is read as adding by a naive learner but as appending a program step by a learner whose theory already includes a memory. The new selector is retrieval. A cue, like a key's label or the display, is coded and recognized by a memory through the same retriever interface the arithmetic model used to recognize problems. Whatever's called up has a view stored with it, and that view is applied. Because it's the same interface, what we proved about memories applies. A cue confused with another brings in the other's view. And a memory with the law of determinacy brings in one view per cue, the same every time. The memory matrix from the first paper proves EPAM's net has that law at every setting and ACT-R's memory has it at none.

When to revise? The decision cycle, again

Learning by revision	Decision cycle (as in the arithmetic solver)
theory, memory, episodes to explain	the state
accept the prediction	`confirm`: precondition, it's right
a failed prediction	**an impasse**
revise, prompted by a cue	**a repair**
which cues to consult	repair heuristics
don't consult a cue twice per failure	a critic

Not a new control structure: the decision cycle that ran the arithmetic backup.

When to revise? The decision cycle, again

Learning by revision	Decision cycle (as in the arithmetic solver)
theory, memory, episodes to explain	the state
accept the prediction	confirm: precondition, it's right
a failed prediction	**an impasse**
revise, prompted by a cue	**a repair**
which cues to consult	repair heuristics
don't consult a cue twice per failure	a critic

Not a new control structure: the decision cycle that ran the arithmetic backup.

And when to revise? The arithmetic model's backup was a problem solver running on CogLib's decision cycle, with impasses and repairs, in the tradition of repair theory and Soar. Revising a theory when a prediction fails has exactly that shape. The state is the theory, the memory and the episodes still to explain. Confirming a prediction is an operator whose precondition is that the prediction is right. A failed prediction is an impasse. Revising the theory, prompted by some cue, is the repair. Which cues to consult are the repair heuristics, and not consulting the same cue twice for one failure is a critic. So the learning loop is not a new control structure. It's the same decision cycle.

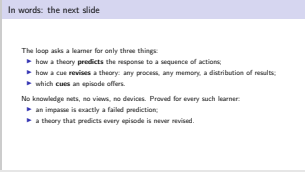
In words: the next slide

The loop asks a learner for only three things:

- how a theory **predicts** the response to a sequence of actions;
- how a cue **revises** a theory: any process, any memory, a distribution of results;
- which **cues** an episode offers.

No knowledge nets, no views, no devices. Proved for every such learner:

- an impasse is exactly a failed prediction;
- a theory that predicts every episode is never revised.

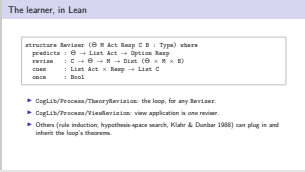


The next slide shows the Lean definition, so here it is in words. The loop asks a learner for only three things: how a theory predicts the response to a sequence of actions, how a cue revises a theory, which can be any process with any memory and gives a distribution over results, and which cues an episode offers. There's nothing about knowledge nets, views or devices at this level. And for every such learner we prove two things: being at an impasse is exactly having a failed prediction, and a theory that predicts every episode is never revised, whatever the revision process is.

The learner, in Lean

```
structure Reviser (Θ M Act Resp C B : Type) where
  predicts : Θ → List Act → Option Resp
  revise   : C → Θ → M → Dist (Θ × M × B)
  cues     : List Act × Resp → List C
  once     : Bool
```

- `CogLib/Process/TheoryRevision`: the loop, for any `Reviser`.
- `CogLib/Process/ViewRevision`: view application is *one* reviser.
- Others (rule induction; hypothesis-space search, Klahr & Dunbar 1988) can plug in and inherit the loop's theorems.



Here's the definition. A reviser has a theory type, a memory type, actions, responses, cues, and whatever a revision brings in. It supplies predicts, revise, cues, and a flag for whether a cue may be consulted twice. The loop is defined for any reviser. View application is one of them, in its own module. Others could be added, rule induction, say, or search in a hypothesis space guided by experiments, and they'd inherit the loop's theorems without reproving them.

A phenomenon that transfers

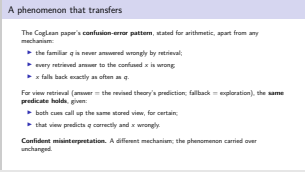
The CogLean paper's **confusion-error pattern**, stated for arithmetic, apart from any mechanism:

- the familiar q is never answered wrongly by retrieval;
- every retrieved answer to the confused x is wrong;
- x falls back exactly as often as q.

For view retrieval (answer = the revised theory's prediction; fallback = exploration), the **same predicate holds**, given:

- both cues call up the same stored view, for certain;
- that view predicts q correctly and x wrongly.

Confident misinterpretation. A different mechanism; the phenomenon carried over unchanged.



Now the phenomena. The first paper defined the confusion-error pattern for arithmetic, as a predicate that names no mechanism: the familiar problem is never answered wrongly by retrieval, every retrieved answer to the confused one is wrong, and the confused one falls back exactly as often as the familiar one. For view retrieval, read the answer as the revised theory's prediction, and the fallback as exploring. Then the same predicate holds, given that both cues call up the same view for certain and that view predicts one correctly and the other wrongly. That's confident misinterpretation. The mechanism is different: there's no associative search with a criterion here. We deliberately didn't force view retrieval into the arithmetic mechanism. What transfers is the phenomenon, and that's exactly what stating phenomena apart from mechanisms is for.

New phenomena: garden paths and perseveration

- **Garden path**: the first interpretation adopted is later replaced.
- **Perseveration**: the same interpretation adopted at two successive failures.

Both stated over the sequence of interpretations, naming no mechanism.

Perseveration, for a class of memories. For *every* memory with determinacy (unchanged by revising): revisions by a given cue bring in one and the same retrieval, every time, from every theory.

- A learner who keeps consulting that cue keeps re-applying a failed interpretation: a relative of *Einstellung* (Luchins 1942).
- ACT-R's memory lacks the law and can escape.
- Sufficient, not necessary; and not a claim that people's memories have the law.

New phenomena: garden paths and perseveration

- **Garden path**: the first interpretation adopted is later replaced.
- **Perseveration**: the same interpretation adopted at two successive failures.

Both stated over the sequence of interpretations, naming no mechanism.

Perseveration, for a class of memories. For *every* memory with determinacy (unchanged by revising): revisions by a given cue bring in one and the same retrieval, every time, from every theory.

- A learner who keeps consulting that cue keeps re-applying a failed interpretation: a relative of *Einstellung* (Luchins 1942).
- ACT-R's memory lacks the law and can escape.
- Sufficient, not necessary; and not a claim that people's memories have the law.

There are two new phenomena, both defined over the sequence of interpretations a learner brings in. A garden path: the first interpretation is later replaced. Perseveration: the same interpretation is adopted at two failures in a row. Then a class-level result. For every memory with the law of determinacy, provided revising doesn't change it, revisions by a given cue always bring in the same thing. So a learner who keeps consulting that cue keeps re-applying an interpretation that already failed. That's a formal relative of Einstellung, and the persistence comes from a property of memory, not of the device or the view. ACT-R's memory lacks the law and can escape. As always: the law is sufficient, not shown necessary, and nothing here says people's memories have it.

What was shared, and what was new

CogLib component	Arithmetic	Concepts
Exact distributions; `bind`	yes	yes
Retriever interface; EPAM; ACT-R	yes	yes
Laws of memory	yes	yes
`ConfusionErrorPattern`	yes	yes
Decision cycle, impasses, repairs	solver	revision loop
Knowledge nets; view application	—	yes
New, general: theory execution, view retrieval, the revision loop, garden paths, perseveration	—	new

Reused by *citing proofs*, not by imitation. The device is data in its own package.

Concept Formation by View Application

What was shared, and what was new

What was shared, and what was new

CogLib component	Arithmetic	Concepts
Exact distributions; `bind`	yes	yes
Retriever interface; EPAM; ACT-R	yes	yes
Laws of memory	yes	yes
`ConfusionErrorPattern`	yes	yes
Decision cycle, impasses, repairs	solver	revision loop
Knowledge nets; view application	—	yes
New, general: theory execution, view retrieval, the revision loop, garden paths, perseveration	—	new

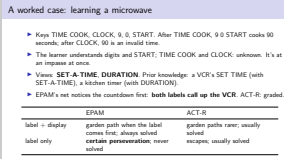
Reused by *citing proofs*, not by imitation. The device is data in its own package.

Here's the accounting. Exact distributions, the retriever interface with EPAM and ACT-R, the laws of memory, the confusion-error pattern and the decision cycle were all used in arithmetic and all used again here. Knowledge nets and view application come from the library's concept work. And what's new, theory execution, view retrieval, the revision loop and the two phenomena, went into CogLib in general form, with nothing about devices in it. Two points. The shared parts were reused by citing their proofs, not by writing look-alikes. And the microwave you're about to see is just data in its own package.

A worked case: learning a microwave

- Keys TIME COOK, CLOCK, 9, 0, START. After TIME COOK, 9 0 START cooks 90 seconds; after CLOCK, 90 is an invalid time.
- The learner understands digits and START; TIME COOK and CLOCK: unknown. It's at an impasse at once.
- Views: **SET-A-TIME**, **DURATION**. Prior knowledge: a VCR's SET TIME (with SET-A-TIME), a kitchen timer (with DURATION).
- EPAM's net notices the countdown first: **both labels call up the VCR**. ACT-R: graded.

	EPAM	ACT-R
label + display	garden path when the label comes first; always solved	garden paths rarer; usually solved
label only	**certain perseveration**; never solved	escapes; usually solved


A worked case: learning a microwave

- Keys TIME COOK, CLOCK, 9, 0, START. After TIME COOK, 9 0 START cooks 90 seconds; after CLOCK, 90 is an invalid time.
- The learner understands digits and START; TIME COOK and CLOCK: unknown. It's at an impasse at once.
- Views: **SET-A-TIME**, **DURATION**. Prior knowledge: a VCR's SET TIME (with SET-A-TIME), a kitchen timer (with DURATION).
- EPAM's net notices the countdown first: **both labels call up the VCR**. ACT-R: graded.

	EPAM	ACT-R
label + display	garden path when the label comes first; always solved	garden paths rarer; usually solved
label only	**certain perseveration**; never solved	escapes; usually solved

The worked case is a generic microwave. After TIME COOK, nine zero START cooks for ninety seconds; after CLOCK, ninety is an invalid time. The learner understands the digits and START but not TIME COOK or CLOCK, so its first prediction fails. There are two views, set-a-time and duration, and the learner's prior knowledge is a VCR's SET TIME key and a kitchen timer. EPAM's net happens to notice the countdown first, so both key labels call up the VCR. ACT-R retrieves gradedly. With the label and the display as cues, EPAM goes down a garden path whenever it looks at the label first, set-a-time, fails, then duration, and always ends up right. With the label only, EPAM perseverates for certain, exactly as the class-level theorem says, and never gets it. ACT-R's graded retrieval usually escapes. Exact values are in the paper's appendix; they're witnesses, not predictions about people.

What it says about concept formation

- **Which prior concept is recruited is a memory question.** Properties of memory become predictions: determinacy ⇒ perseveration; confusion ⇒ confident misinterpretation.
- **When to revise is problem solving**: the impasse structure, connecting theory change to repair theory and Soar.
- **Views act on types.** TIME COOK and CLOCK share a type, so one view changes both; no single view gets both right. Fixing that means *splitting* the type: differentiation, in the conceptual-change sense (Carey; Thagard). Not yet in CogLean.

What it says about concept formation

- **Which prior concept is recruited is a memory question.** Properties of memory become predictions: determinacy → perseveration; confusion → confident misinterpretation.
- **When to revise is problem solving**: the impasse structure, connecting theory change to repair theory and Soar.
- **Views act on types.** TIME COOK and CLOCK share a type; so one view changes both; no single view gets both right. Fixing that means *splitting* the type: differentiation, in the conceptual-change sense (Carey; Thagard). Not yet in CogLean.

What does formalizing this say about concept formation? Three things. First, which prior concept gets recruited is a memory question, so properties of memory turn into predictions about concept formation: determinacy gives perseveration, confusion gives confident misinterpretation. Second, when to revise isn't a separate learning mechanism; it's the impasse structure of problem solving. Third, views act on types. In the microwave, TIME COOK and CLOCK share a type, so any view changes both together, and no single view gets both right. Getting both right means splitting the type, which is a formal version of what accounts of conceptual change call differentiation. CogLean doesn't do that yet, and it's one of the most interesting next steps.

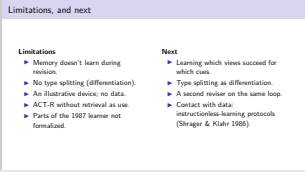
Limitations, and next

Limitations

- Memory doesn't learn during revision.
- No type splitting (differentiation).
- An illustrative device; no data.
- ACT-R without retrieval as use.
- Parts of the 1987 learner not formalized.

Next

- Learning which views succeed for which cues.
- Type splitting as differentiation.
- A second reviser on the same loop.
- Contact with data: instructionless-learning protocols (Shrager & Klahr 1986).

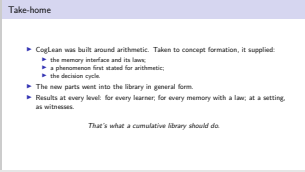


The limitations are real. Memory doesn't learn while the theory is being revised, so a cue never comes to retrieve a better view. Type splitting isn't formalized. The device and the prior knowledge are illustrative, and there's no data. ACT-R appears without retrieval changing memory, as in the first paper. And several mechanisms of the 1987 learner aren't formalized. Next: learning which views succeed for which cues; type splitting as differentiation; a second, different reviser on the same loop, to see the loop's results transfer between revision processes the way the confusion pattern transferred between domains; and contact with data, starting with the instructionless-learning protocols.

Take-home

- CogLean was built around arithmetic. Taken to concept formation, it supplied:
  - the memory interface and its laws;
  - a phenomenon first stated for arithmetic;
  - the decision cycle.
- The new parts went into the library in general form.
- Results at every level: for every learner; for every memory with a law; at a setting, as witnesses.

That's what a cumulative library should do.



To sum up. CogLean was built around arithmetic. Taken to a very different domain, concept formation, it supplied the memory interface and its laws, a phenomenon first written down for arithmetic, and the decision cycle. What had to be added went into the library in general form, ready for the next domain. And the results came out at the levels the first paper laid out: for every learner, for every memory with a law, and at a particular setting, as a witness. The library is still small. But that's the behavior a cumulative library should show.